All the Joins in SQL- Visualized and Simplified

Spardha Follow 8 min read · Jun 16, 2021

85

A JOIN command in SQL is used to combine data or rows from two or more tables based on a common field between them. Primarily, there are four main joins in SQL:

1. INNER JOIN

2. LEFT JOIN

3. RIGHT JOIN

4. FULL JOIN

However, in this article, I'm also going to discuss the following joins and operators:

1. CROSS JOIN

2. SELF JOIN

3. UNION

4. UNION ALL

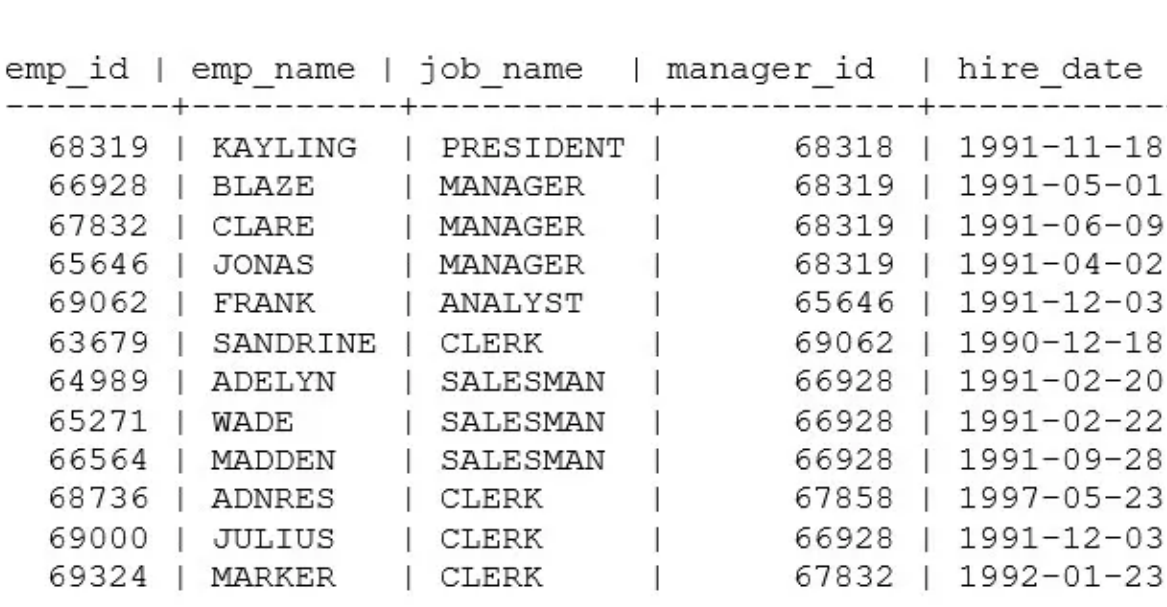
5. INTERSECT

6. EXCEPT

Every topic is supplemented by a visualisation. At the end, there is also a table that mentions the differences between JOIN and UNION; INNER JOIN and INTERSECT. *All commands have been performed in MySQL.*

I'll be using the following two tables in the examples to explain each topic:

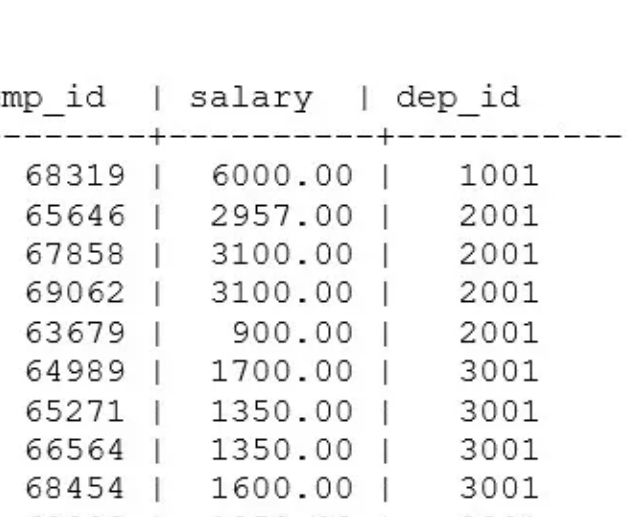
```
employees
```



```
emp_id | emp_name | job_name  | manager_id  | hire_date
--------+----------+-----------+------------+------------
  68319 | KAYLING  | PRESIDENT |      68318 | 1991-11-18
  66928 | BLAZE    | MANAGER   |      68319 | 1991-05-01
  67832 | CLARE    | MANAGER   |      68319 | 1991-06-09
  65646 | JONAS    | MANAGER   |      68319 | 1991-04-02
  69062 | FRANK    | ANALYST   |      65646 | 1991-12-03
  63679 | SANDRINE | CLERK     |      69062 | 1990-12-18
  64989 | ADELYN   | SALESMAN  |      66928 | 1991-02-20
  65271 | WADE     | SALESMAN  |      66928 | 1991-02-22
  66564 | MADDEN   | SALESMAN  |      66928 | 1991-09-28
  68736 | ADNRES   | CLERK     |      67858 | 1997-05-23
  69000 | JULIUS   | CLERK     |      66928 | 1991-12-03
  69324 | MARKER   | CLERK     |      67832 | 1992-01-23
```

employees table | Image by Author

```
salary
```

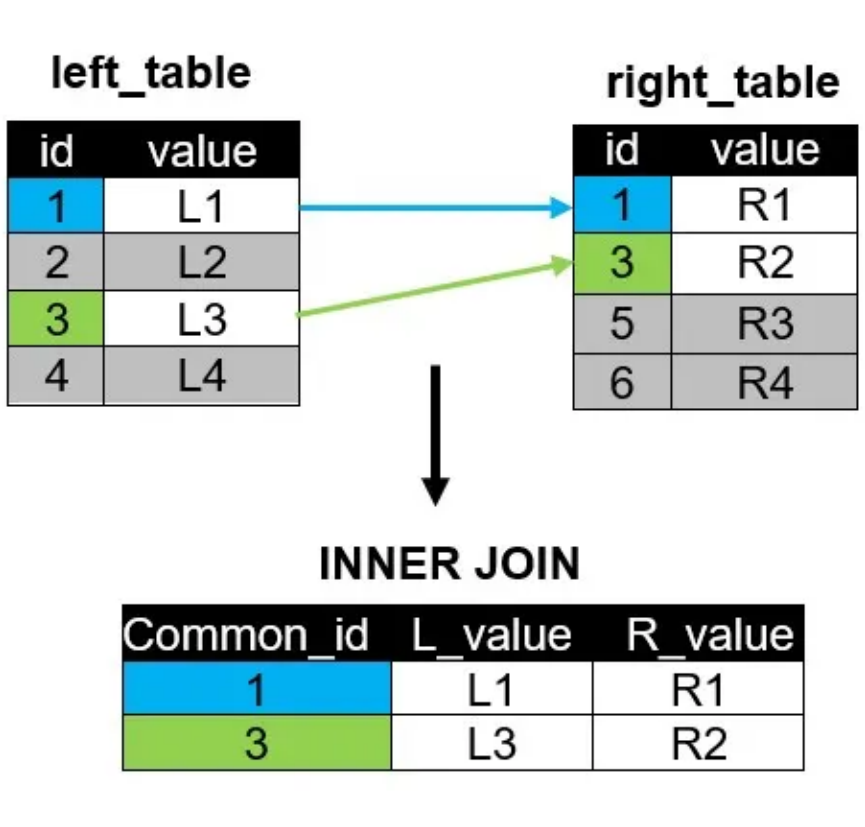


```
emp_id  | salary   | dep_id
--------+----------+----------
  68319 |  6000.00 |   1001
  65646 |  2957.00 |   2001
  67858 |  3100.00 |   2001
  69062 |  3100.00 |   2001
  63679 |   900.00 |   2001
  64989 |  1700.00 |   3001
  65271 |  1350.00 |   3001
  66564 |  1350.00 |   3001
  68454 |  1600.00 |   3001
  69000 |  1050.00 |   3001
```

salary table | Image by Author

1. INNER JOIN

The INNER JOIN keyword will create the resultant table by combining all rows from both the tables where the condition satisfies i.e., value of the common field will be same.



id	value
1	L1
2	L2
3	L3
4	L4

id	value
1	R1
3	R2
5	R3
6	R4

Common_id	L_value	R_value
1	L1	R1
3	L3	R2

INNER JOIN Working Process | Image by Author

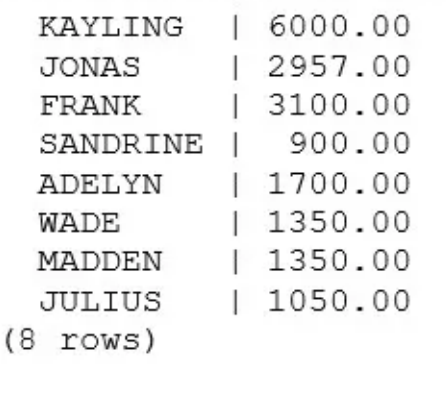
Example:

Let's query the employee names (`emp_name`) and their respective salaries (`salary`) for *values of* `emp_id` *that are same in both the tables.*

Note: `employees.emp_id` and `salary.emp_id` are the matching key columns.

```
SELECT e.emp_name, s.salary FROM employees AS e
INNER JOIN
salary AS s
ON e.emp_id=s.emp_id;
```

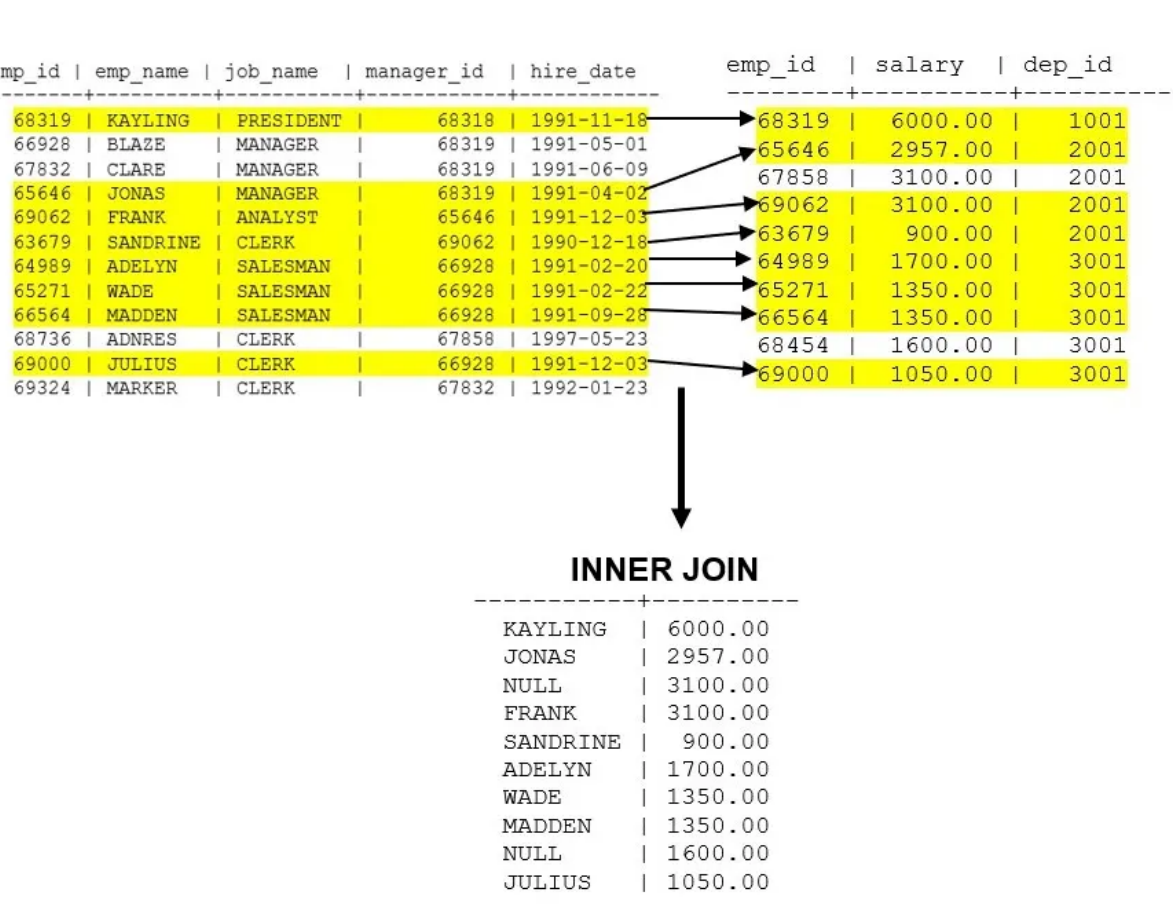
We get the following output:



```
-----------+----------
 KAYLING   | 6000.00
 JONAS     | 2957.00
 FRANK     | 3100.00
 SANDRINE  |  900.00
 ADELYN    | 1700.00
 WADE      | 1350.00
 MADDEN    | 1350.00
 JULIUS    | 1050.00
(8 rows)
```

Output for INNER JOIN Command | Image by Author

As you can see, the output is data containing all rows of the values of `emp_id` that are present in both employees and salary tables. The following image outlines the working process of INNER JOIN in this example.



```
emp_id | emp_name | job_name  | manager_id | hire_date
--------+----------+-----------+------------+------------
 68319 | KAYLING  | PRESIDENT |      68318 | 1991-11-18
 66928 | BLAZE    | MANAGER   |      68319 | 1991-05-01
 67832 | CLARE    | MANAGER   |      68319 | 1991-06-09
 65646 | JONAS    | MANAGER   |      68319 | 1991-04-02
 69062 | FRANK    | ANALYST   |      65646 | 1991-12-03
 63679 | SANDRINE | CLERK     |      69062 | 1990-12-18
 64989 | ADELYN   | SALESMAN  |      66928 | 1991-02-20
 65271 | WADE     | SALESMAN  |      66928 | 1991-02-22
 66564 | MADDEN   | SALESMAN  |      66928 | 1991-09-28
 68736 | ADNRES   | CLERK     |      67858 | 1997-05-23
 69000 | JULIUS   | CLERK     |      66928 | 1991-12-03
 69324 | MARKER   | CLERK     |      67832 | 1992-01-23
```

```
emp_id  | salary   | dep_id
--------+----------+----------
68319 |  6000.00 |   1001
65646 |  2957.00 |   2001
67858 |  3100.00 |   2001
69062 |  3100.00 |   2001
63679 |   900.00 |   2001
64989 |  1700.00 |   3001
65271 |  1350.00 |   3001
66564 |  1350.00 |   3001
68454 |  1600.00 |   3001
69000 |  1050.00 |   3001
```

```
-----------+----------
 KAYLING   | 6000.00
 JONAS     | 2957.00
 NULL      | 3100.00
 FRANK     | 3100.00
 SANDRINE  |  900.00
 ADELYN    | 1700.00
 WADE      | 1350.00
 MADDEN    | 1350.00
 NULL      | 1600.00
 JULIUS    | 1050.00
```

INNER JOIN Example | Image by Author

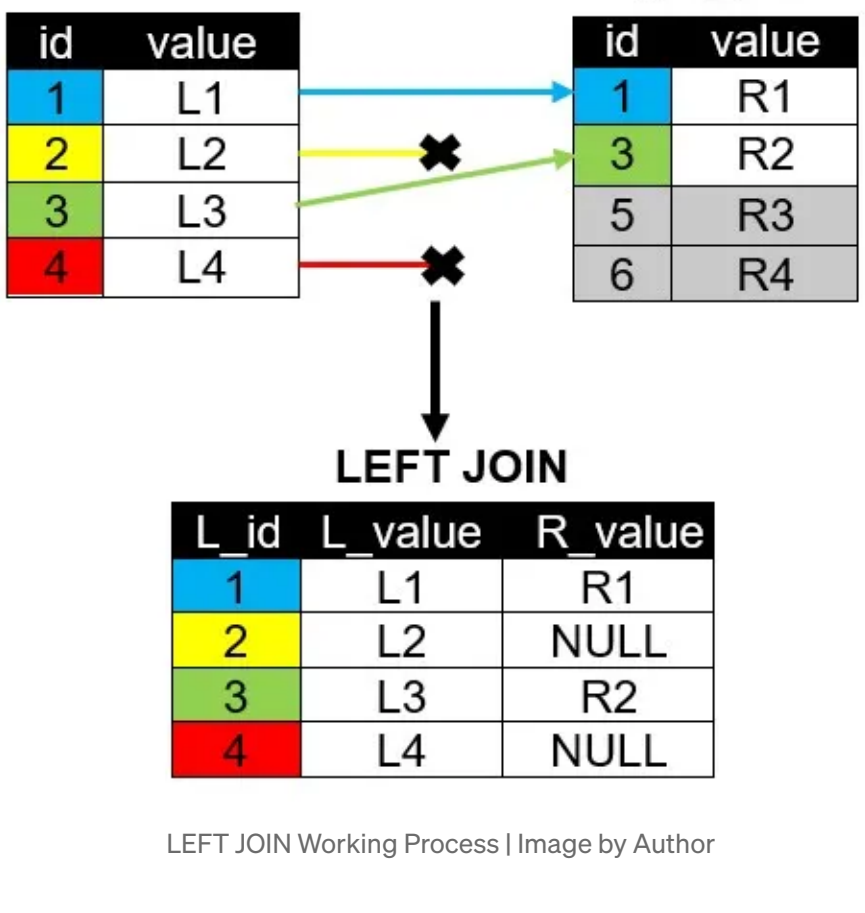
Since the name of matching key column is same in both the tables, i.e. `emp_id`, we can also employ the USING command instead of ON. Note that you have to use parenthesis () after the USING keyword to mention the name of the matching key column.

```
SELECT e.emp_name, s.salary FROM employees AS e
INNER JOIN
salary AS s
USING (emp_id);
```

We will get the same output.

2. LEFT JOIN

The LEFT JOIN keyword will create a table that contains all the rows of the left side table and matching rows of the right-side table. The rows for which there is no matching row on right side, the resultant table will contain `NULL`. LEFT JOIN is also known as LEFT OUTER JOIN.



id	value
1	L1
2	L2
3	L3
4	L4

id	value
1	R1
3	R2
5	R3
6	R4

L_id	L_value	R_value
1	L1	R1
2	L2	NULL
3	L3	R2
4	L4	NULL

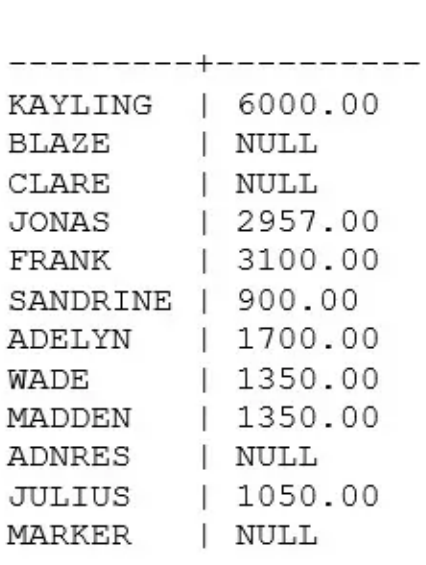
LEFT JOIN Working Process | Image by Author

Example:

Let's query employee names (`emp_name`) and their corresponding salaries (`salary`) from the employees and salary tables respectively.

```
SELECT e.emp_name, s.salary FROM employees AS e
LEFT JOIN
salary AS s
ON e.emp_id=s.emp_id;
```

We get the following output:

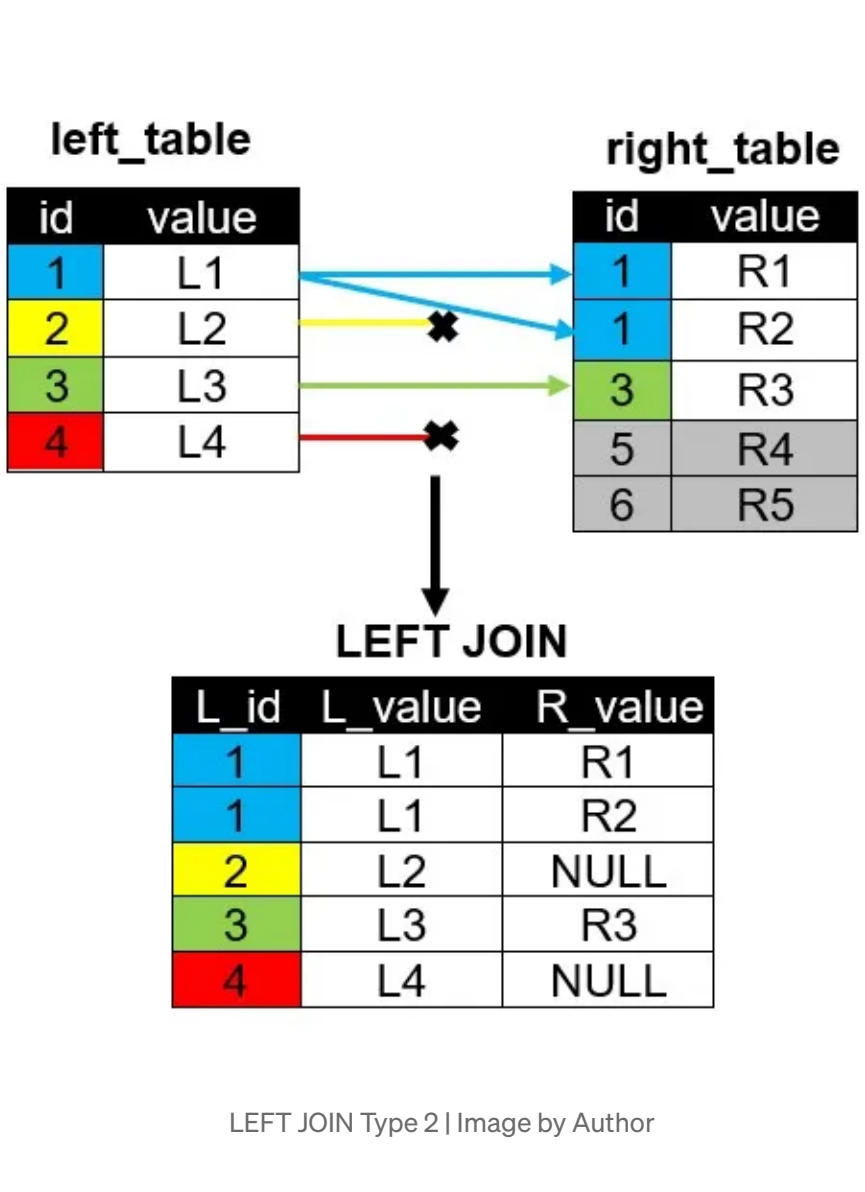


```
---------+----------
KAYLING  | 6000.00
BLAZE    | NULL
CLARE    | NULL
JONAS    | 2957.00
FRANK    | 3100.00
SANDRINE | 900.00
ADELYN   | 1700.00
WADE     | 1350.00
MADDEN   | 1350.00
ADNRES   | NULL
JULIUS   | 1050.00
MARKER   | NULL
```

Output for LEFT JOIN Command | Image by Author

Since there are some `emp_id`s which are present on the left side table but not the right side table, few salary values are missing or `NULL`.

What happens if there is more than one row of data on the right side table, for the same matching id on the left side table? All the corresponding values in the right side table get printed.



id	value
1	L1
2	L2
3	L3
4	L4

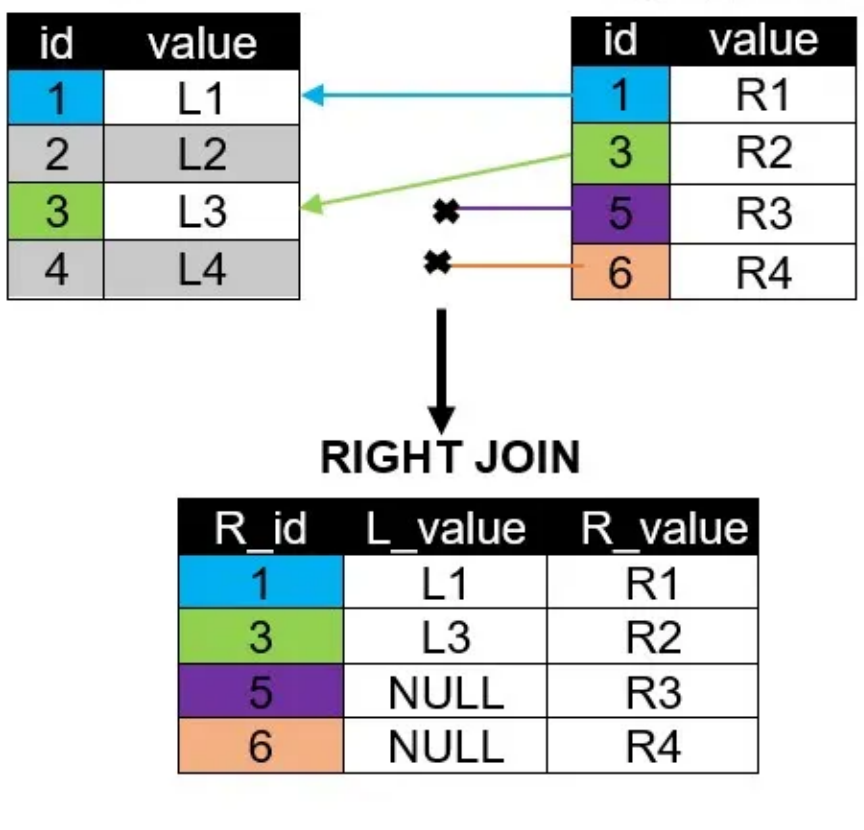
id	value
1	R1
1	R2
3	R3
5	R4
6	R5

L_id	L_value	R_value
1	L1	R1
1	L1	R2
2	L2	NULL
3	L3	R3
4	L4	NULL

LEFT JOIN Type 2 | Image by Author

3. RIGHT JOIN

The RIGHT JOIN keyword will create resultant table that contains all the rows of the right-side table and matching rows of the left-side table. The rows for which there is no matching row on the left side, the resultant table will contain `NULL`. RIGHT JOIN is also known as RIGHT OUTER JOIN.



id	value
1	L1
2	L2
3	L3
4	L4

id	value
1	R1
3	R2
5	R3
6	R4

R_id	L_value	R_value
1	L1	R1
3	L3	R2
5	NULL	R3
6	NULL	R4

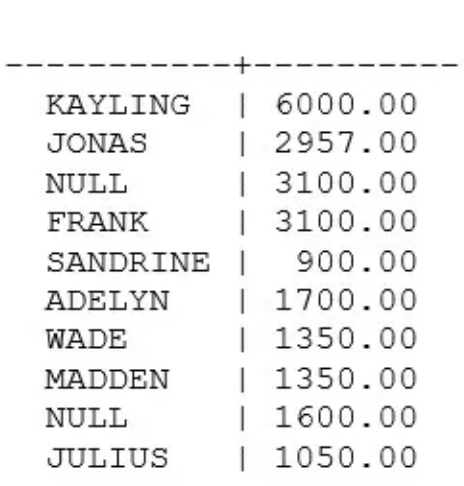
RIGHT JOIN Working Process | Image by Author

Example:

Let's query salaries (`salary`) and their corresponding employee names (`emp_name`) from the salary and employees tables respectively.

```
SELECT e.emp_name, s.salary FROM employees AS e
RIGHT JOIN
salary AS s
ON e.emp_id=s.emp_id;
```

We get the following output:

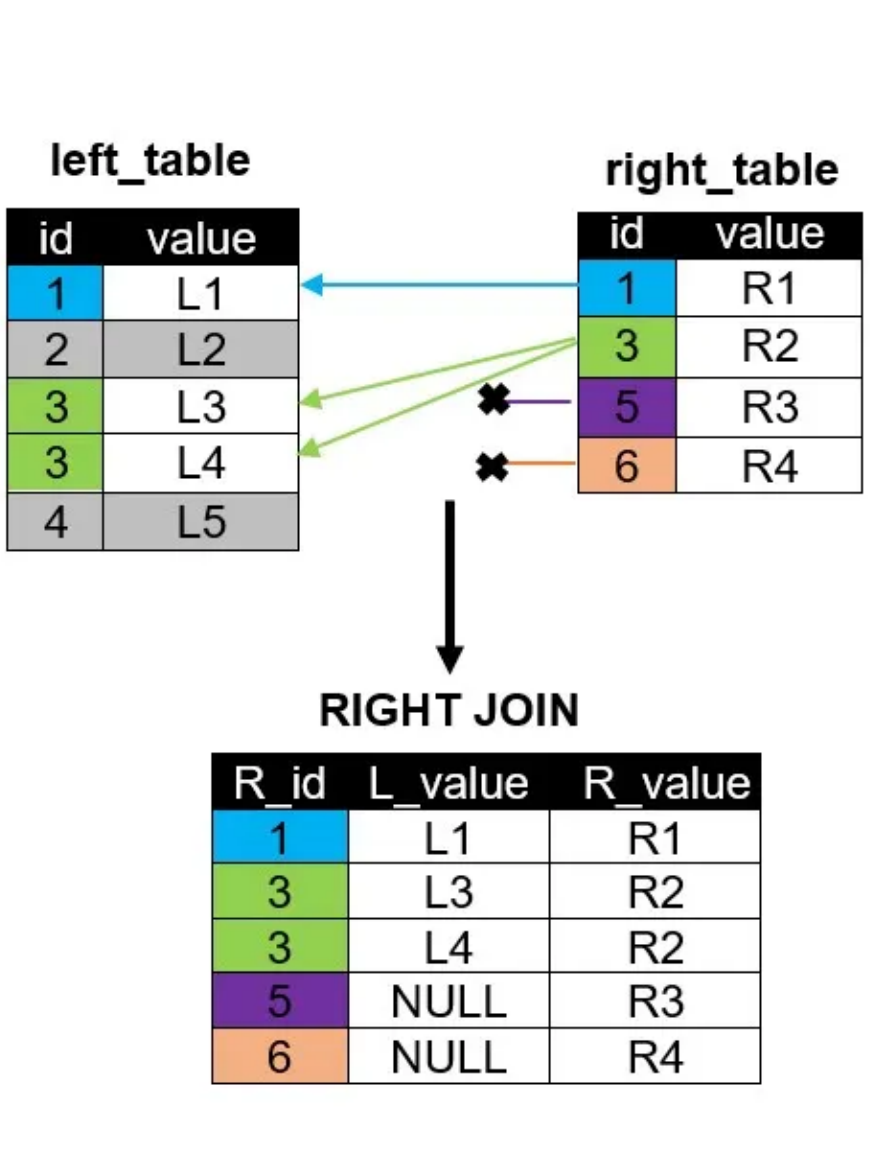


```
-----------+----------
 KAYLING   | 6000.00
 JONAS     | 2957.00
 NULL      | 3100.00
 FRANK     | 3100.00
 SANDRINE  |  900.00
 ADELYN    | 1700.00
 WADE      | 1350.00
 MADDEN    | 1350.00
 NULL      | 1600.00
 JULIUS    | 1050.00
```

Output for RIGHT JOIN Command | Image by Author

Since there are some `emp_id`s which are present on the right side table (`salary`) but not the left side table (`employees`), few employee names are missing or `NULL`.

What happens if there is more than one row of data on the left side table, for the same matching id on the right side table? Similar to the explanation of this case in LEFT JOIN, all the corresponding values in the left side table get printed.



id	value
1	L1
2	L2
3	L3
3	L4
4	L5

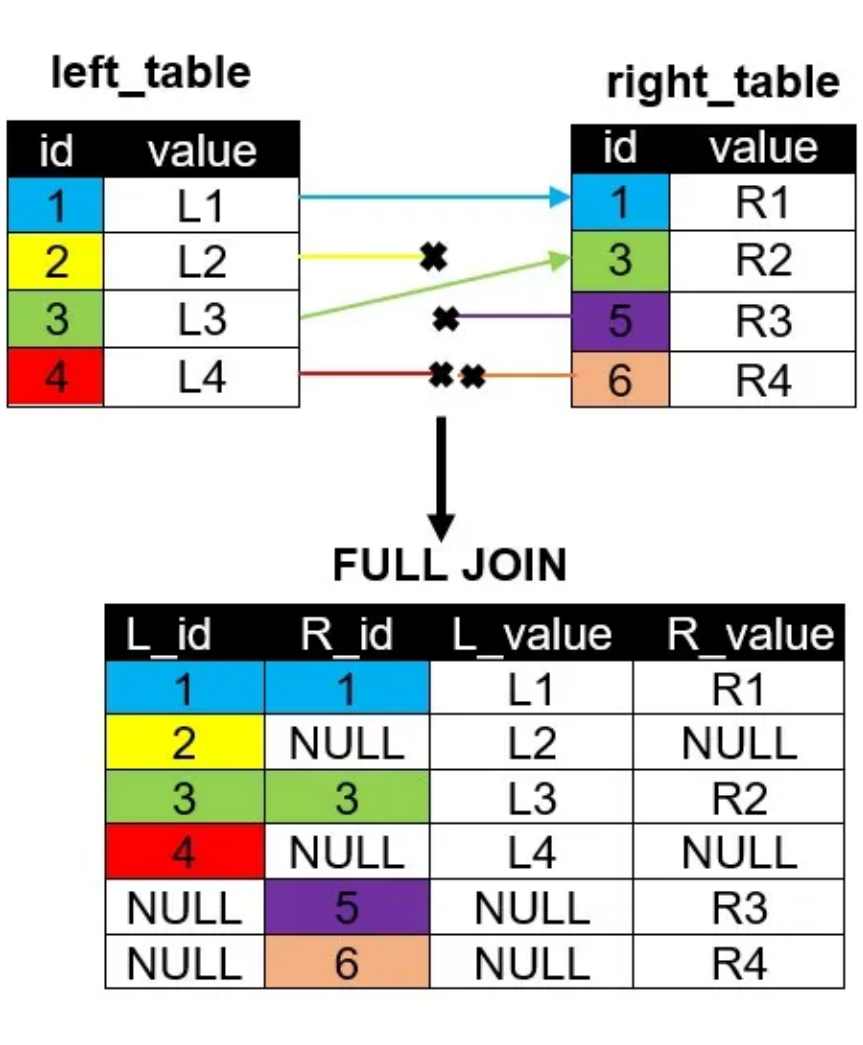
id	value
1	R1
3	R2
5	R3
6	R4

R_id	L_value	R_value
1	L1	R1
3	L3	R2
3	L4	R2
5	NULL	R3
6	NULL	R4

RIGHT JOIN Type 2 | Image by Author

4. FULL JOIN

FULL JOIN keyword will create the resultant table by combining result of both LEFT JOIN and RIGHT JOIN. The table will contain all the rows from both the tables. The rows for which there is no matching, the result-set will contain `NULL` values. FULL JOIN is also known as FULL OUTER JOIN.



id	value
1	L1
2	L2
3	L3
4	L4

id	value
1	R1
3	R2
5	R3
6	R4

L_id	R_id	L_value	R_value
1	1	L1	R1
2	NULL	L2	NULL
3	3	L3	R2
4	NULL	L4	NULL
NULL	5	NULL	R3
NULL	6	NULL	R4

FULL JOIN Working Process | Image by Author

Example

Get Spardha's stories in your inbox

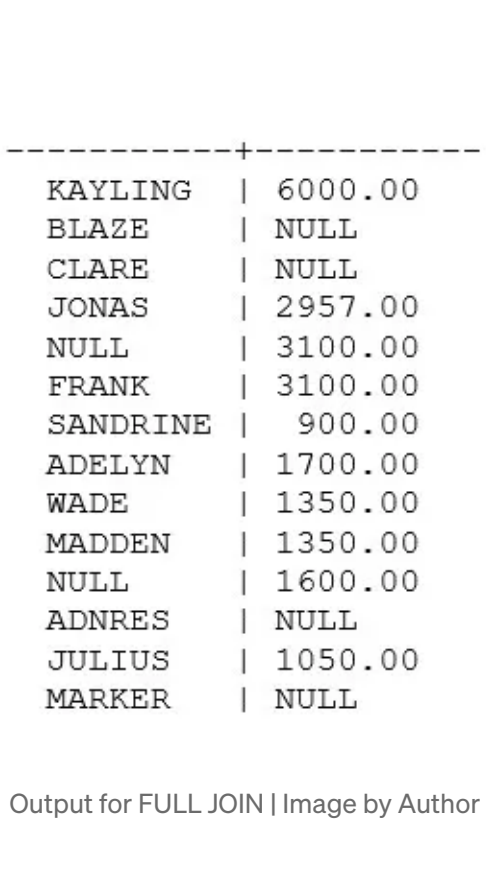
Join Medium for free to get updates from this writer.

Enter your email Subscribe

Let's query *all* employee names (`emp_name`) and *all* salaries (`salary`) from both the given tables.

```
SELECT e.emp_name, s.salary FROM employees AS e
FULL JOIN
salary AS s
ON e.emp_id=s.emp_id;
```

We get the following output:



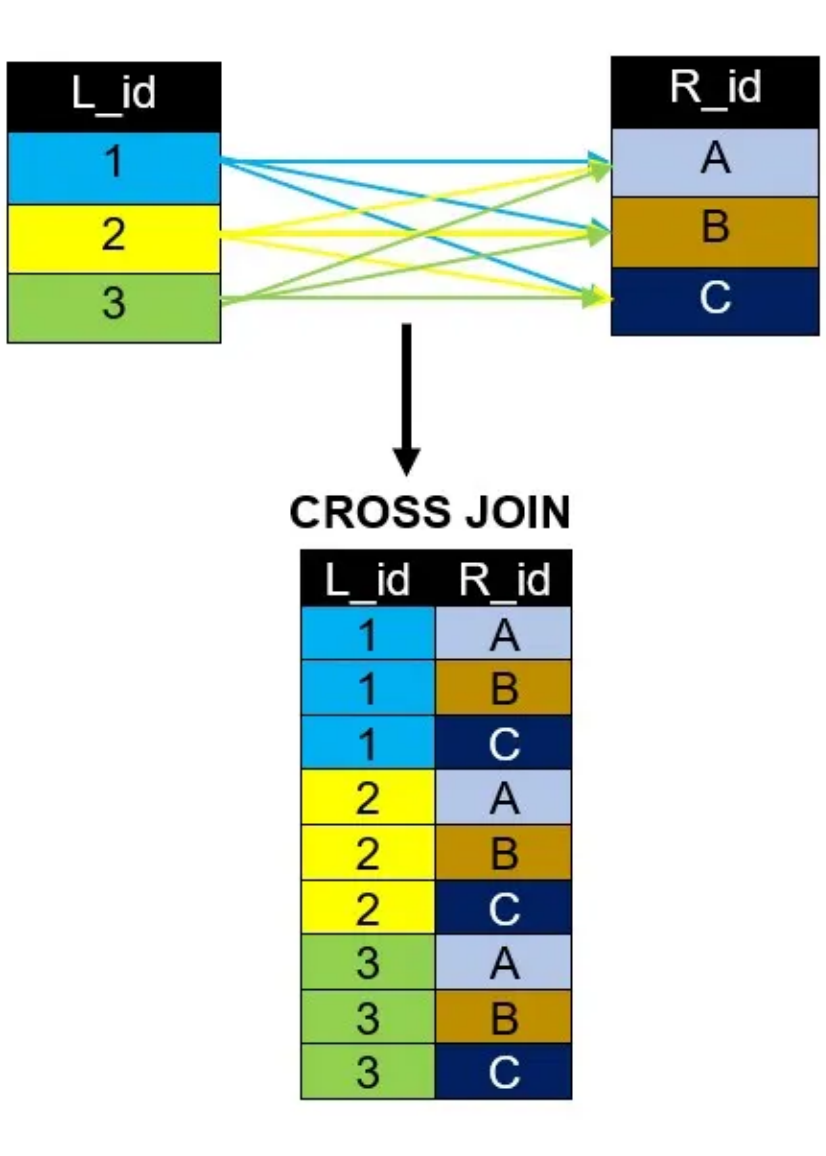
```
-----------+-----------
 KAYLING   | 6000.00
 BLAZE     | NULL
 CLARE     | NULL
 JONAS     | 2957.00
 NULL      | 3100.00
 FRANK     | 3100.00
 SANDRINE  |  900.00
 ADELYN    | 1700.00
 WADE      | 1350.00
 MADDEN    | 1350.00
 NULL      | 1600.00
 ADNRES    | NULL
 JULIUS    | 1050.00
 MARKER    | NULL
```

Output for FULL JOIN | Image by Author

Now that we've covered the four basic joins in SQL, let's move on to the more uncommon ones.

5. CROSS JOIN

CROSS JOIN keyword is used to combine each row of the left table with each row of the right table. It is also known as the Cartesian join since it returns the Cartesian product of the sets of rows from the joined tables.



L_id
1
2
3

R_id
A
B
C

L_id	R_id
1	A
1	B
1	C
2	A
2	B
2	C
3	A
3	B
3	C

CROSS JOIN Working Process | Image by Author

Unlike other joins, CROSS JOIN doesn't require a matching key column as all possible combinations are printed. I won't illustrate CROSS JOIN with an example using the given tables because the information would be inaccurate, i.e. a single employee will have multiple salaries, which is not possible. However, to give a brief idea about the keyword, an input code has been mentioned below.

```
SELECT * FROM employees AS e
CROSS JOIN
salary AS s;
```

6. SELF JOIN

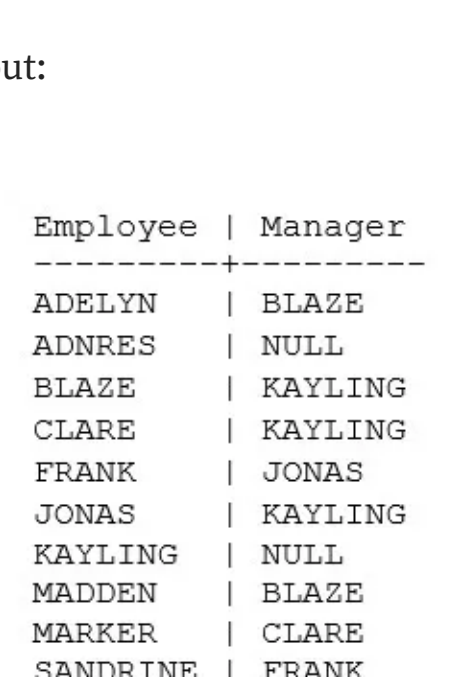
The SELF JOIN keyword will create resultant table by joining a table to itself. Each row of a table is combined with itself *and* with every other row of the table. SELF JOIN is used to extract specific queries and hence does not have a universal working process. At least one table must be renamed in the query.

Example:

Let's query employee's name and their manager's name through SELF JOIN.

```
SELECT a1.emp_name AS Employee, a2.emp_name AS Manager
FROM employees AS a1
JOIN employees AS a2
ON a1.manager_id=a2.emp_id
ORDER BY a1.emp_name ASC;
```

We get the following output:



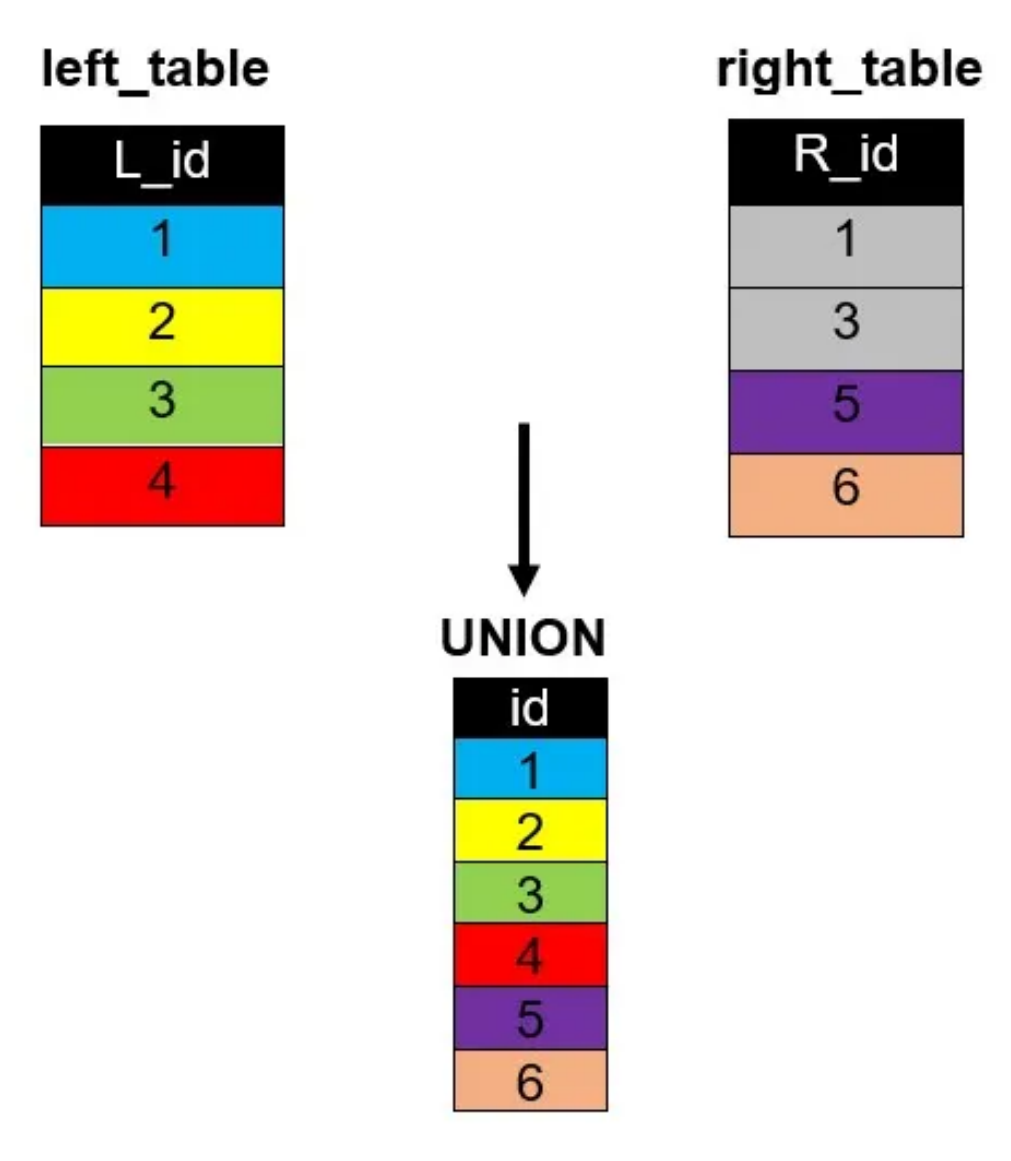
```
Employee | Manager
---------+---------
ADELYN   | BLAZE
ADNRES   | NULL
BLAZE    | KAYLING
CLARE    | KAYLING
FRANK    | JONAS
JONAS    | KAYLING
KAYLING  | NULL
MADDEN   | BLAZE
MARKER   | CLARE
SANDRINE | FRANK
```

SELF JOIN Example | Image by Author

Now that we've covered all the JOINs, let's move on to Operators in SQL.

7. UNION

The UNION keyword is used to combine two separate SELECT statements and produce the resultant table as a merger of both the SELECT statements. The fields to be used in both the select statements must be in same order, same quantity and same data type.

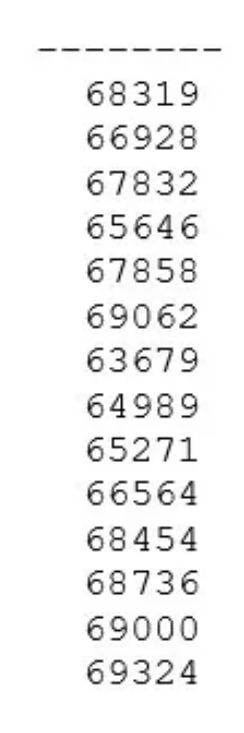


UNION Working Process | Image by Author

Example

```
SELECT emp_id FROM employees
UNION
SELECT emp_id FROM salary;
```

We get the following output:

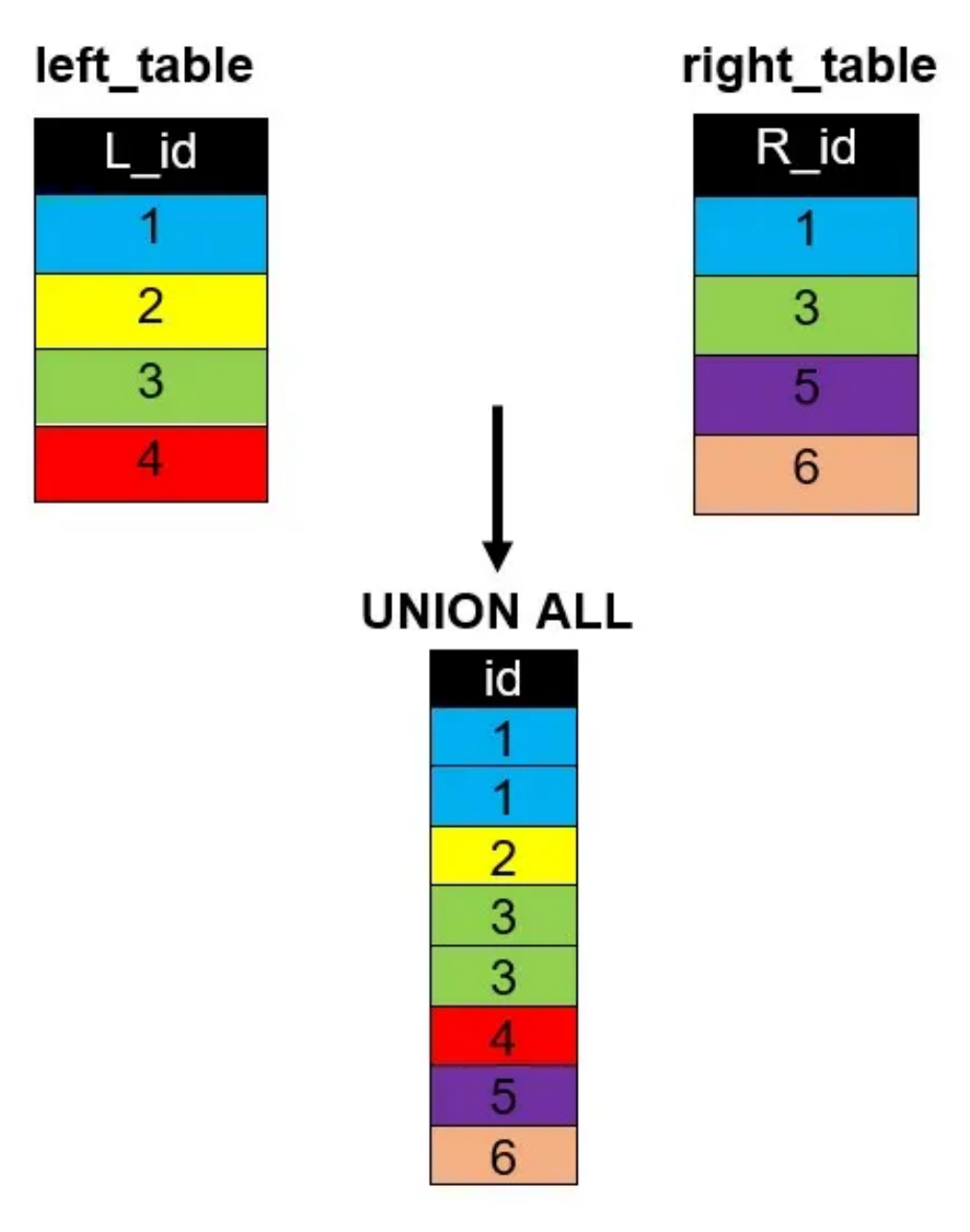


```
--------
  68319
  66928
  67832
  65646
  67858
  69062
  63679
  64989
  65271
  66564
  68454
  68736
  69000
  69324
```

Output for UNION | Image by Author

8. UNION ALL

The UNION ALL keyword is *also* used to combine two separate SELECT statements and produce the resultant table as a merger of both the SELECT statements. As mentioned for UNION, the fields to be used in both the select statements must be in same order, same quantity and same data type.



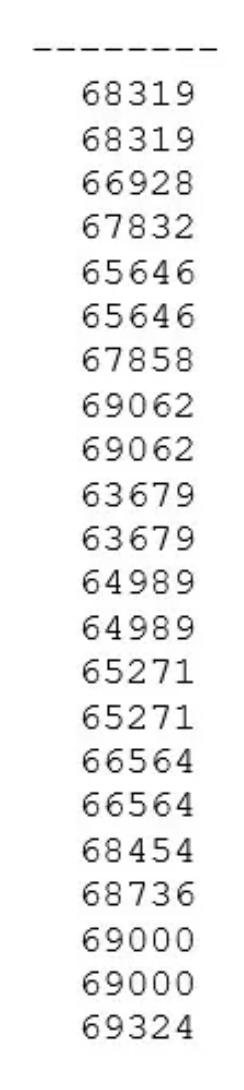
UNION ALL Working Process | Image by Author

The only difference between UNION and UNION ALL is that they return distinct and duplicate values respectively. As a result, if you want to query distinct values, use UNION. If you also want to include the duplicate entries, use UNION ALL.

Example:

```
SELECT emp_id FROM employees
UNION ALL
SELECT emp_id FROM salary;
```

We get the following output:

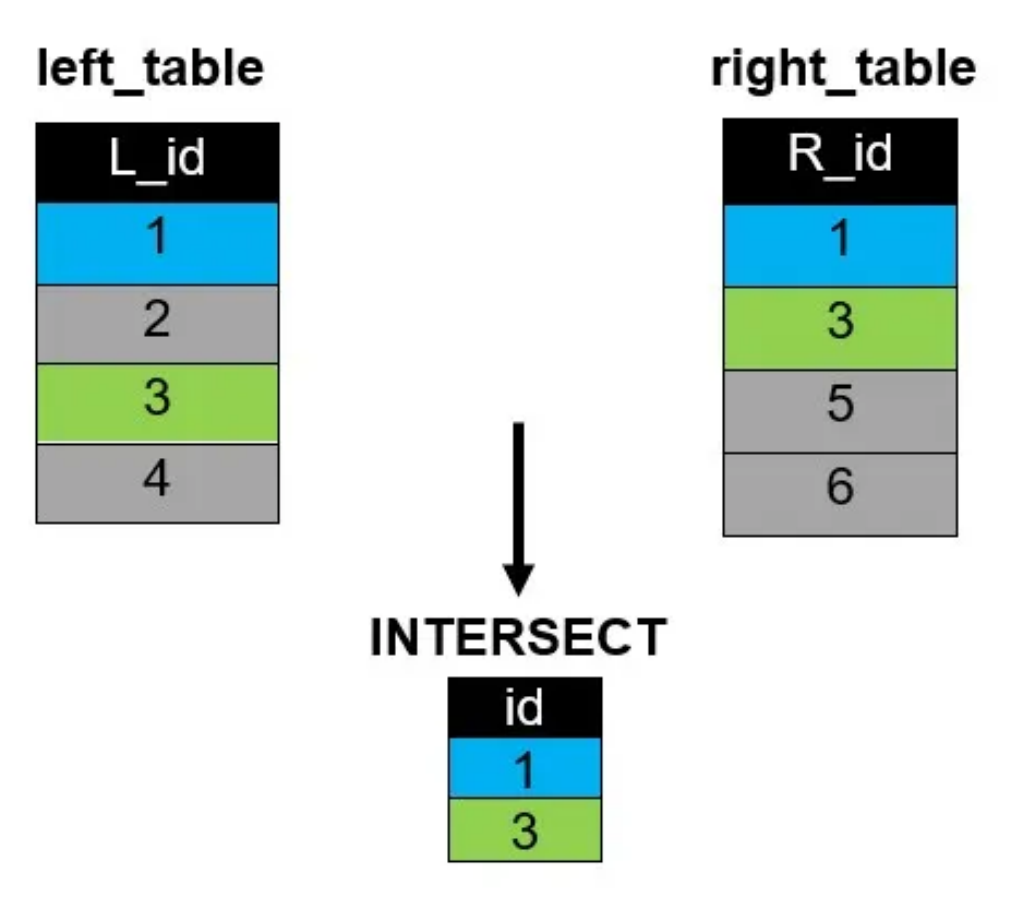


```
--------
  68319
  68319
  66928
  67832
  65646
  65646
  67858
  69062
  69062
  63679
  63679
  64989
  64989
  65271
  65271
  66564
  66564
  68454
  68736
  69000
  69000
  69324
```

Output for UNION ALL | Image by Author

9. INTERSECT

INTERSECT keyword provides the result of the cohesion of two SELECT statements. The resultant table will contain all the rows which are common to both the SELECT statements. It will *not* return any duplicate values.

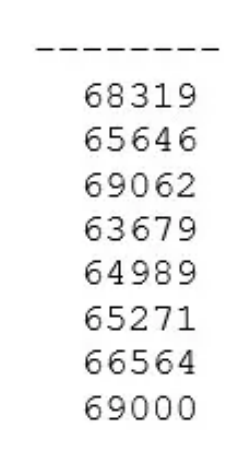


INTERSECT Working Process | Image by Author

Example:

```
SELECT emp_id FROM employees
INTERSECT
SELECT emp_id FROM salary;
```

We get the following output:

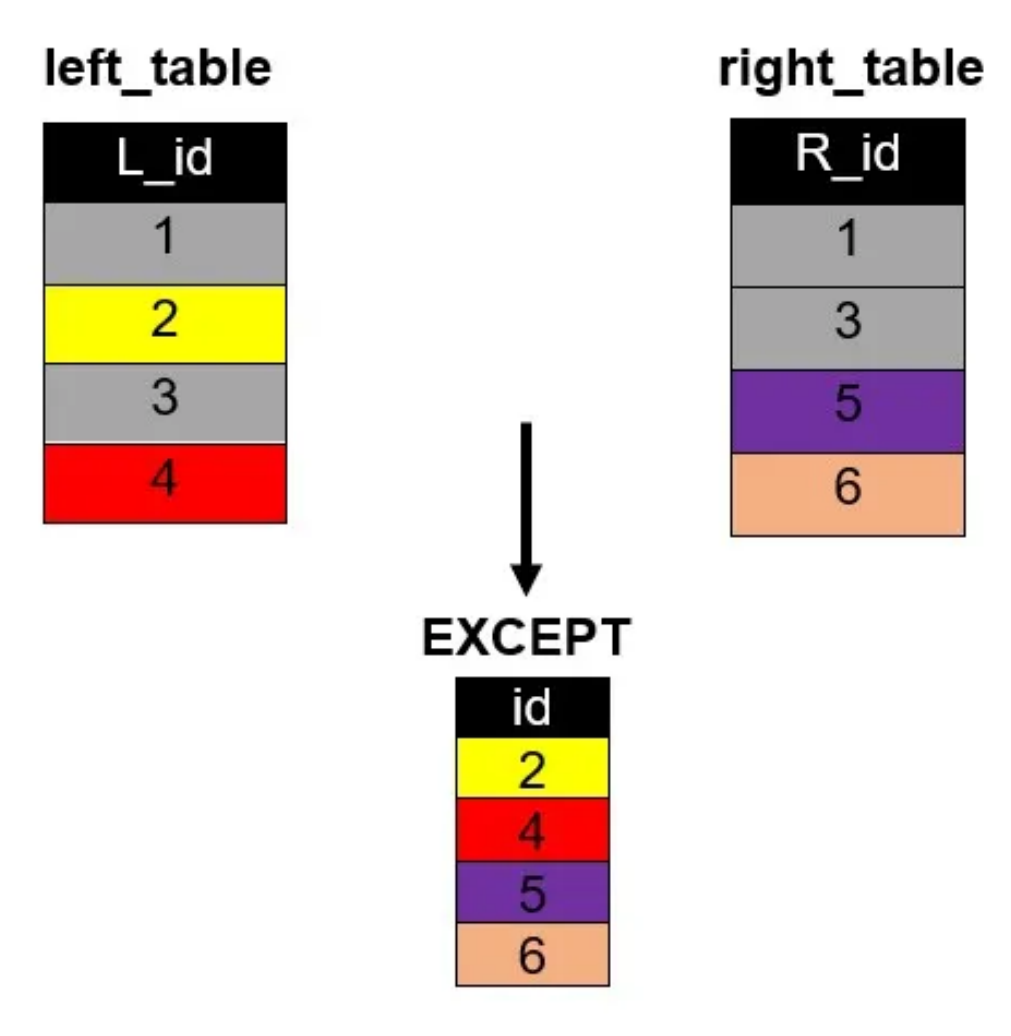


```
--------
  68319
  65646
  69062
  63679
  64989
  65271
  66564
  69000
```

Output for INTERSECT | Image by Author

10. EXCEPT

EXCEPT keyword provides the result of the inconsistency of two SELECT statements. The resultant table will contain all the rows *except* the common rows of the two SELECT statements.

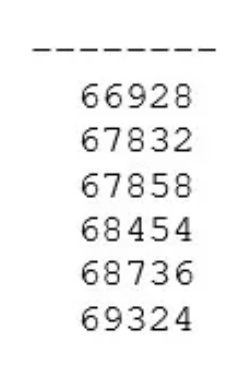


EXCEPT Working Process | Image by Author

Example:

```
SELECT emp_id FROM employees
EXCEPT
SELECT emp_id FROM salary;
```

We get the following output:

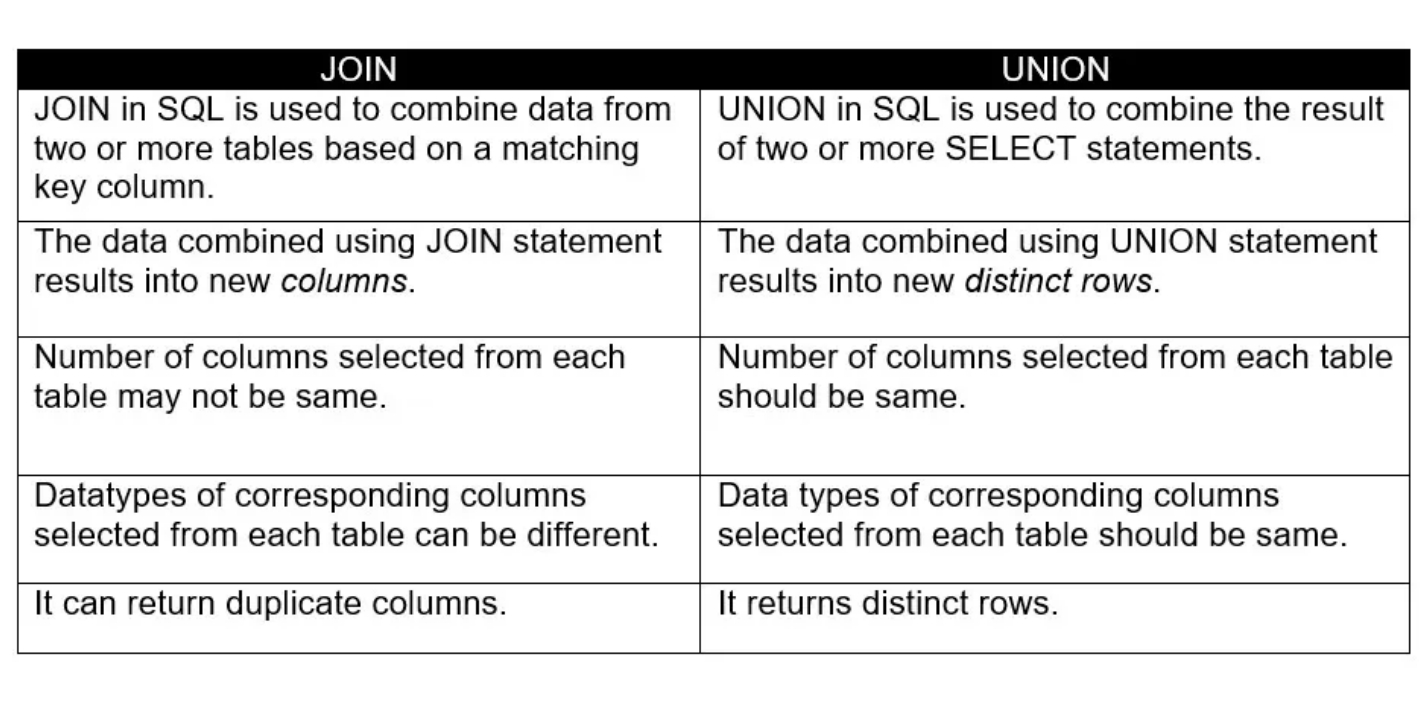


```
--------
  66928
  67832
  67858
  68454
  68736
  69324
```

Output for EXCEPT | Image by Author

Now that we've covered all the JOINs and operators in SQL, let's discuss some differences between topics that may seem similar. For instance, JOIN v/s UNION and INNER JOIN v/s INTERSECT.

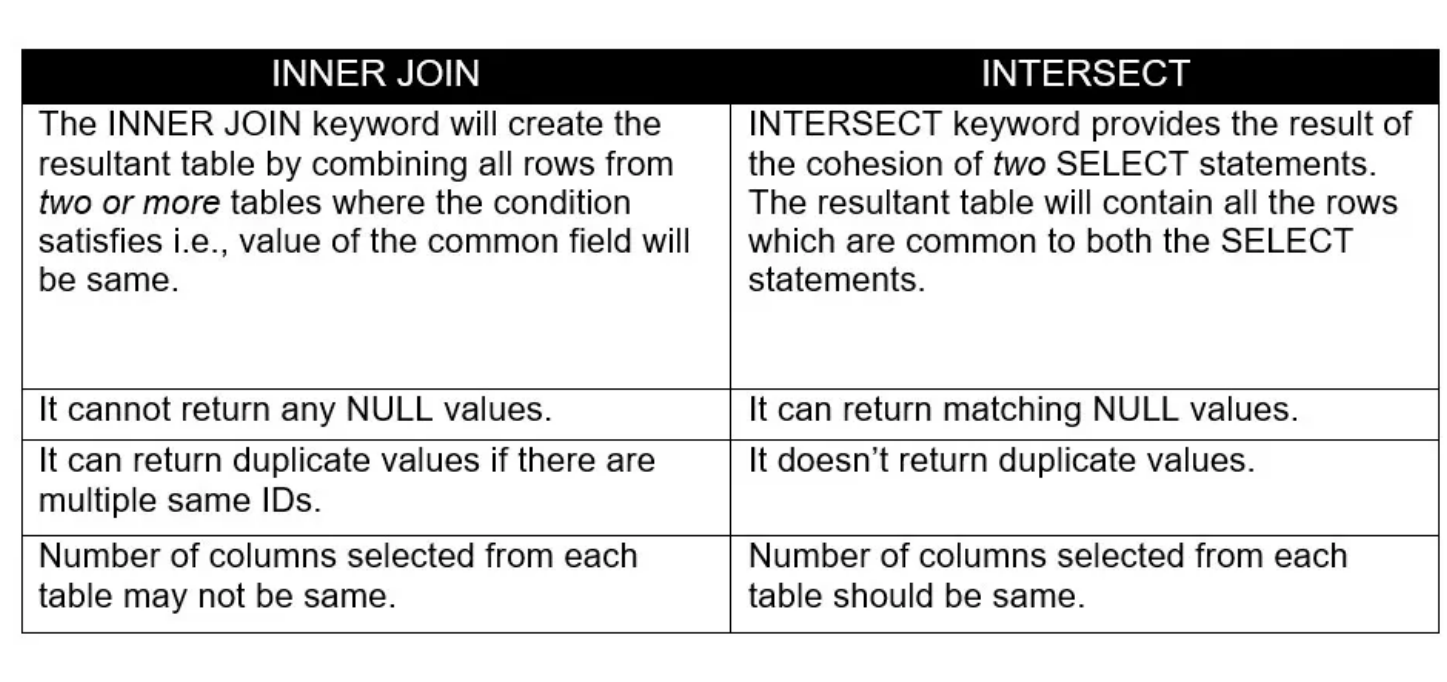
JOIN v/s UNION



JOIN	UNION
JOIN in SQL is used to combine data from two or more tables based on a matching key column.	UNION in SQL is used to combine the result of two or more SELECT statements.
The data combined using JOIN statement results into new *columns*.	The data combined using UNION statement results into new *distinct rows*.
Number of columns selected from each table may not be same.	Number of columns selected from each table should be same.
Datatypes of corresponding columns selected from each table can be different.	Data types of corresponding columns selected from each table should be same.
It can return duplicate columns.	It returns distinct rows.

JOIN and UNION differences | Image by Author

INNER JOIN v/s INTERSECT



INNER JOIN	INTERSECT
The INNER JOIN keyword will create the resultant table by combining all rows from *two or more* tables where the condition satisfies i.e., value of the common field will be same.	INTERSECT keyword provides the result of the cohesion of *two* SELECT statements. The resultant table will contain all the rows which are common to both the SELECT statements.
It cannot return any NULL values.	It can return matching NULL values.
It can return duplicate values if there are multiple same IDs.	It doesn't return duplicate values.
Number of columns selected from each table may not be same.	Number of columns selected from each table should be same.

INNER JOIN and INTERSECT differences | Image by Author

I hope that this article clarified your concepts on the various types of JOINs in SQL; as well as some operators. This is a fairly important subject with respect to Data Science Interviews and thus, it's essential to be familiar with the aforementioned topics.

Sql MySQL Data Science Interview Programming

85

Written by Spardha

Follow

71 followers · 5 following

Columbia University | Passionate about Data Science

No responses yet

Write a response

What are your thoughts?

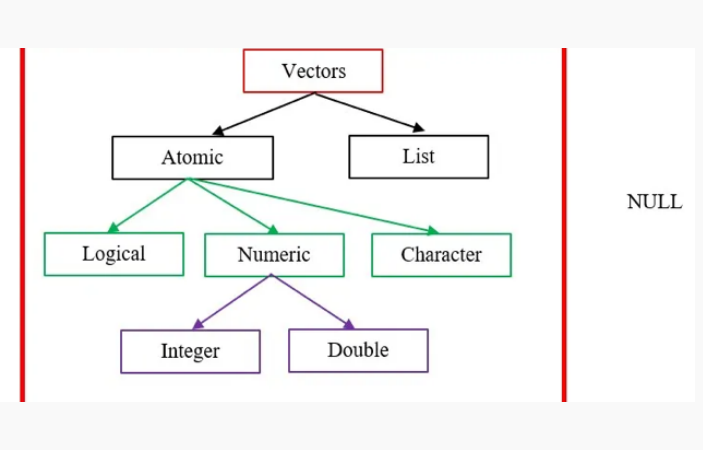
More from Spardha

In Analytics Vidhya by Spardha

7 Types of Discrete Probability Distributions and Their…

Binomial, Multinomial, Bernoulli, Negative Binomial, Poisson Distribution and more.

Aug 20, 2021 103 1



In Analytics Vidhya by Spardha

The Ultimate Beginner's Guide to Data Types in R

NULL vs NA vs NaN; [ ] vs [[ ]]- I cover it all.

May 23, 2021 101

In Python in Plain English by Spardha

Building a Basic Chatbot with Python's NLTK Library

A Step-by-Step Guide on Chatbots with NLTK

May 20, 2021 65

See all from Spardha

Recommended from Medium

The Analytics Edge 🌟

You Don't Know SQL Until You've Written These 10 Queries

Go beyond SELECT *—Master real SQL by writing the queries that separate pros from…

Aug 2 199 6

Rizqi Mulki

SQL Query Optimization: Tips That Actually Work

Stop writing slow queries that make your users (and your database) cry

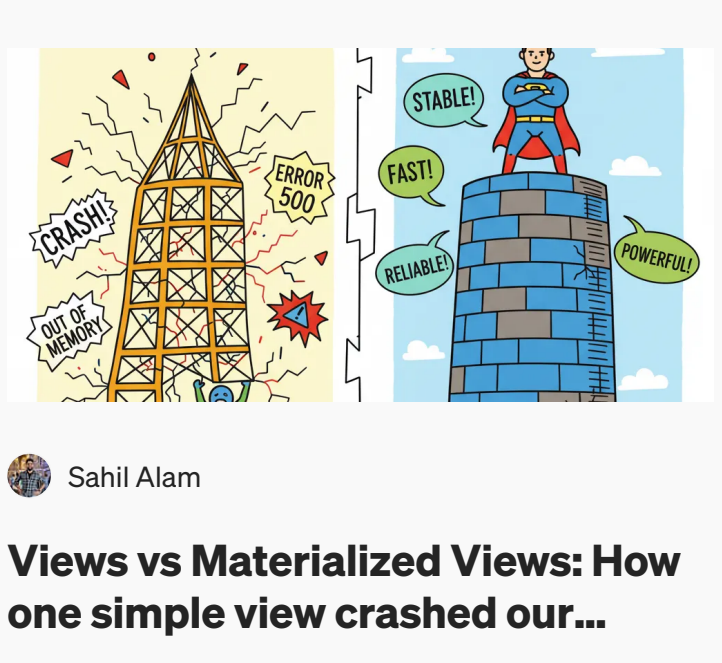
Aug 10 2

Abdul Qadir | SQL & AI

📝 How I Learned SQL in a Real Fintech Job—Not From a Course

No roadmap, no tutorial. Just real deadlines, real data, and real mistakes. That's how I…

Aug 5 214

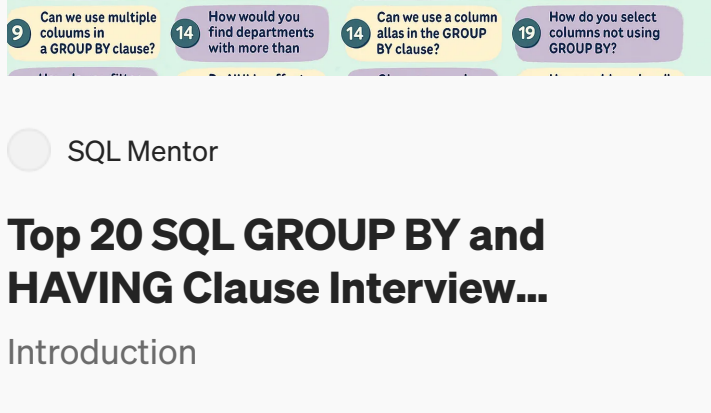

CRASH!
OUT OF MEMORY
ERROR 500
STABLE!
FAST!
RELIABLE!
POWERFUL!

Sahil Alam

Views vs Materialized Views: How one simple view crashed our…

Picture this: It's 9 AM on a Monday morning. Coffee in hand, I'm feeling pretty good about…

Aug 29 6

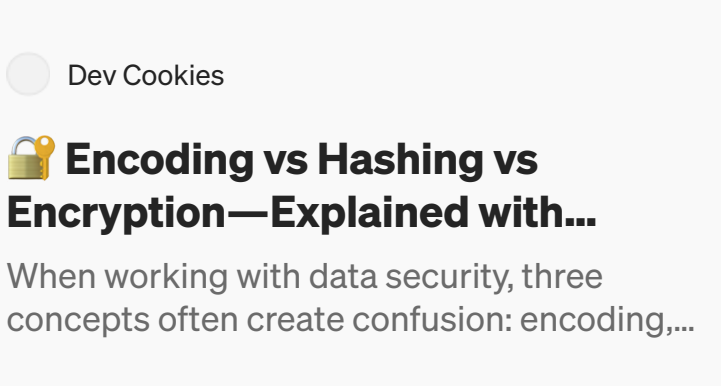

SQL GROUP BY and HAVING Clause
INTERVIEW QUESTIONS

SQL Mentor

Top 20 SQL GROUP BY and HAVING Clause Interview…

Introduction

Jun 13 6



Dev Cookies

🔐 Encoding vs Hashing vs Encryption—Explained with…

When working with data security, three concepts often create confusion: encoding,…

Aug 22 7

See more recommendations